

## HBASE – APUNTE

HBase está diseñada para grandes trabajos. A primera vista, HBase parece una base de datos relacional, organiza la información en “buckets” a las que llama “tables”, que contienen “cells” que aparecen en la intersección filas y columnas. Sin embargo, las tablas no se comportan como relaciones, las filas no funcionan y las columnas son completamente variables.

HBase es una base de datos para grandes volúmenes de información. Si se usa para megabytes o unos pocos gigas, estás haciendo algo mal.

Tiene: versionado, compresión, garbage collector y tablas en memoria.

### INTRODUCING HBASE

Es una base de datos orientada en columnas. Está basada en Bigtable (propietaria de Google). Se caracteriza por ser resistente a fallos (WAL Write-Ahead-Logging y configuración distribuida).

### DAY 1: CRUD AND TABLE ADMINISTRATION

HBase tiene 3 modos principales, nosotros nos vamos a basar principalmente en el primero:

- Standalone mode: Una sola máquina actuando sola.
- Pseudo-distributed mode: Un único nodo fingiendo ser un cluster.
- Fully distributed mode: Un cluster de nodos trabajando en conjunto.

### CREATING A TABLE

Una tabla en HBase es un mapa de mapas (key/value map, similar a JSON).

Las keys son strings arbitrarios que mapean a una fila de datos, que es, en sí misma, un mapa cuyos keys se llaman columns y sus valores se guardan como un array de bytes no interpretado:

|     | row keys | column family<br>"color"                            | column family<br>"shape"         |
|-----|----------|-----------------------------------------------------|----------------------------------|
| row | "first"  | "red": "#F00"<br>"blue": "#00F"<br>"yellow": "#FF0" | "square": "4"                    |
| row | "second" |                                                     | "triangle": "3"<br>"square": "4" |
|     |          |                                                     |                                  |

En python:

```
hbase_table = {    # Table
    'row1': {      # Row key
        'cf1:col1': 'value1', # Column family, column,
        and value
        'cf1:col2': 'value2',
        'cf2:col1': 'value3'
    },
    'row2': {      # More row data
    }
}
queried_value = hbase_table['row1']['cf1:col1'] # 'value1'
```

---

Para crear una tabla en HBase:

```
hbase> create 'wiki', 'text'
```

### INSERTING, UPDATING AND RETRIEVING DATA

Para añadir información a la tabla que creamos usamos el comando put

```
hbase> put 'wiki', 'Home', 'text:', 'Welcome to the wiki!'
```

Para solicitar la info de la fila 'Home' usamos get.  
Requiere 2 parámetros, el nombre de la tabla y la llave de la fila:

```
hbase> get 'wiki', 'Home', 'text:'
```

El output será:

```
COLUMN    CELL
text: timestamp=1295774833226, value=Welcome to the wiki!
1 row(s) in 0.0590 seconds
```

Cuando se modifica el valor en la misma celda, el valor antiguo se guarda indexado por su timestamp (versionado).

Para retornar todas las filas en la tabla usamos scan (¡Usar con precaución!):

```
hbase> scan 'wiki'
```

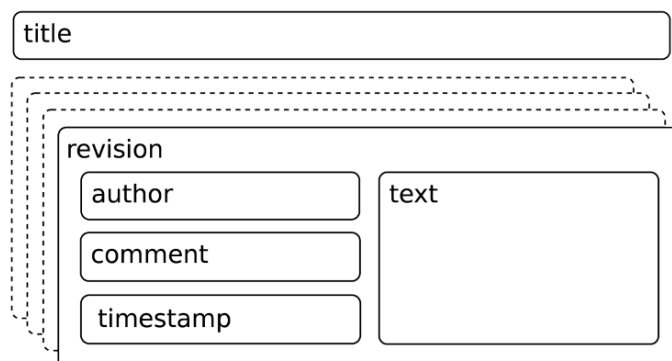
### PUTTING AND GETTING

Los comandos `put` y `get` nos permiten especificar timestamps de forma explícita. También se puede especificar otro valor entero de preferencia. Por defecto HBase usa el timestamp más reciente.

### ALTERING TABLES

- In our wiki, a page is uniquely identified by its title.
- A page can have unlimited revisions.
- A revision is identified by its timestamp.
- A revision contains text and optionally a commit comment.
- A revision was made by an author, identified by name.

Visually, our requirements can be sketched as you see in the following figure.



|               | keys<br>(title) | family<br>"text" | family<br>"revision"                |
|---------------|-----------------|------------------|-------------------------------------|
| row<br>(page) | "first page"    | "": "..."        | "author": "..."<br>"comment": "..." |
| row<br>(page) | "second page"   | "": "..."        | "author": "..."<br>"comment": "..." |
|               |                 |                  |                                     |

## DEFAULTS

Para hacer modificaciones, primero hay que dejar “offline” a la tabla (comando disable):

```
hbase> disable 'wiki'
```

Ahora podemos modificar las características de la familia de columnas usando el comando “alter”:

```
hbase> alter 'wiki', { NAME => 'text', VERSIONS =>
hbase*
org.apache.hadoop.hbase.HConstants::ALL_VERSIONS }
```

## ALTERING A TABLE

Las modificaciones en una familia de columnas es sumamente costosa (se crea una nueva columna y se copia toda la información, mientras antes se hagan estos cambios, mejor). Aún con nuestra tabla offline:

```
hbase> alter 'wiki', { NAME => 'revision', VERSIONS =>
hbase*
org.apache.hadoop.hbase.HConstants::ALL_VERSIONS }
```

## MOVING ON

Una vez realizados los cambios, reensablamos la base de datos:

```
hbase> enable 'wiki'
```

## ADDING DATA PROGRAMMATICALLY

A veces, los comandos put pueden ser insuficientes, por lo que podemos usar scripting (usando JRuby, que a su vez utiliza la JVM)

```
hbase/put_multiple_columns.rb
import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'

def jbytes(*args)
  args.map {|arg| arg.to_s.to_java_bytes}
end

table = HTable.new(@hbase.configuration, "wiki")
p = Put.new(*jbytes("Home"))
p.add(*jbytes("text", "", "Hello world"))
p.add(*jbytes("revision", "author", "jimbo"))
p.add(*jbytes("revision", "comment", "my first edit"))
table.put(p)
```

La idea es crear múltiples tablas con el mismo timestamp.

## WHY COLUMN FAMILIES

La principal razón es que se pierde el ajuste de eficiencia de alta granularidad, la otra ventaja es que las familias de columnas están alojadas en diferentes directorios.

Todas las operaciones son atómicas a nivel de filas.

## DAY 2: WORKING WITH BIG DATA

### IMPORTING DATA, INVOKING SCRIPTS

Para correr un script desde la línea de comandos:

```
$ ${HBASE_HOME}/bin/hbase shell <your_script>
[<optional_arguments> ...]
```

Cómo estamos enfocados en “Big Data” vamos a importar artículos de wikipedia:

```

<page>
  <title>Anarchism</title>
  <id>12</id>
  <revision>
    <id>408067712</id>
    <timestamp>2011-01-15T19:28:25Z</timestamp>
    <contributor>
      <username>RepublicanJacobite</username>
      <id>5223685</id>
    </contributor>
    <comment>Undid revision 408057615 by [[Special:Contributions...</comment>
    <text xml:space="preserve">{{Redirect|Anarchist|the fictional character|
...
[[bat-smg:Anarkézmōs]]
  </text>
</revision>
</page>

```

## STREAMING XML

Hay que paresear los XML gigantes de forma de streaming:

```

hbase/basic_xml_parsing.rb
import 'javax.xml.stream.XMLStreamConstants'

factory = javax.xml.stream.XMLInputFactory.newInstance
reader = factory.createXMLStreamReader(java.lang.System.in)

while reader.has_next
  type = reader.next

  if type == XMLStreamConstants::START_ELEMENT
    tag = reader.local_name
    # do something with tag
  elsif type == XMLStreamConstants::CHARACTERS
    text = reader.text
    # do something with text
  elsif type == XMLStreamConstants::END_ELEMENT
    # same as START_ELEMENT
  end
end

```

## STREAMING WIKIPEDIA

```
hbase/import_from_wikipedia.rb
```

```
require 'time'

import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'
import 'javax.xml.stream.XMLStreamConstants'

def jbytes(*args)
  args.map { |arg| arg.to_s.to_java_bytes }
end

factory = javax.xml.stream.XMLInputFactory.newInstance
reader = factory.createXMLStreamReader(java.lang.System.in)

document = nil
buffer = nil
count = 0

table = HTable.new(@hbase.configuration, 'wiki')
table.setAutoFlush(false)

while reader.has_next
  type = reader.next

  if type == XMLStreamConstants::START_ELEMENT
    case reader.local_name
    when 'page' then document = {}
    when '/title|timestamp|username|comment|text/' then buffer = []
    end

  elsif type == XMLStreamConstants::CHARACTERS
    buffer << reader.text unless buffer.nil?

  elsif type == XMLStreamConstants::END_ELEMENT
    case reader.local_name
    when '/title|timestamp|username|comment|text/'
      document[reader.local_name] = buffer.join
    when 'revision'
      key = document['title'].to_java_bytes
      ts = (Time.parse document['timestamp']).to_i

      p = Put.new(key, ts)
      p.add(*jbytes("text", "", document['text']))
      p.add(*jbytes("revision", "author", document['username']))
      p.add(*jbytes("revision", "comment", document['comment']))
      table.put(p)

      count += 1
      table.flushCommits() if count % 10 == 0
      if count % 500 == 0
        puts "#{count} records inserted (#{document['title']})"
      end
    end
  end
end

table.flushCommits()
exit
```

## COMPRESSION AND BLOOM FILTERS

HBase nos permite comprimir la información para acelerar las lecturas:

```
hbase> alter 'wiki', {NAME=>'text', COMPRESSION=>'GZ',  
BLOOMFILTER=>'ROW'}
```

Hay 2 formas de compresión principales: LZ0 y GZ. Normalmente se prefiere usar LZ0. En este caso se usa Gzip por un tema de licencias.

Los bloom filters son estructuras que responden la pregunta ¿He visto esto antes? y evitan queries caras que ya han fracasado previamente.

## INTRODUCTION TO REGIONS AND MONITORING DISK USAGE

Las filas se almacenan en orden, según la llave de la fila. Un chunk es una región de memoria identificada por la starting key (inclusive) y la ending key (excluyente).

En el directorio data/default, en el hbase.rootdir ejecutamos:

```
$ du -h  
4.0K    ./wiki/.tabledesc  
0B      ./wiki/.tmp  
0B      ./wiki/1e157605a0e5a1493e4cc91d7e368b05/.tmp  
0B      ./wiki/1e157605a0e5a1493e4cc91d7e368b05/recovered.edits  
11M     ./wiki/1e157605a0e5a1493e4cc91d7e368b05/revision  
64M     ./wiki/1e157605a0e5a1493e4cc91d7e368b05/text  
75M     ./wiki/1e157605a0e5a1493e4cc91d7e368b05  
75M     ./wiki  
75M     .
```

A tener en cuenta que las modificaciones no se generan automáticamente, sino que primero se registran en un buffer (WAL)

## REGIONAL INTERROGATION



En bases de datos grandes, HBase parte las regiones en múltiples bloques para que se distribuyan mejor entre los nodos.

```
$ du -h
4.0K    ./wiki/.tabledesc
0B      ./wiki/.tmp
0B      ./wiki/48576fdcfcb9b29257fb93d33dbeda90/.tmp
0B      ./wiki/48576fdcfcb9b29257fb93d33dbeda90/recovered.edits
132M    ./wiki/48576fdcfcb9b29257fb93d33dbeda90/revision
145M    ./wiki/48576fdcfcb9b29257fb93d33dbeda90/text
277M    ./wiki/48576fdcfcb9b29257fb93d33dbeda90
0B      ./wiki/bd36620826a14025a35f1fe5e928c6c9/.tmp
0B      ./wiki/bd36620826a14025a35f1fe5e928c6c9/recovered.edits
134M    ./wiki/bd36620826a14025a35f1fe5e928c6c9/revision
113M    ./wiki/bd36620826a14025a35f1fe5e928c6c9/text

247M    ./wiki/bd36620826a14025a35f1fe5e928c6c9
1.0G    ./wiki
1.0G    .
```

Esta información se puede verificar desde el shell:

```
hbase> scan 'hbase:meta', { COLUMNS => ['info:server',
'info:regioninfo'] }
```

(Esta data también puede ser fragmentada en regiones y distribuidas en regiones dentro del servidor).

La asignación de las regiones está definida por un nodo maestro o un “master node”. Es el que gestiona el WAL ante fallos.

### SCANNING ONE TABLE TO BUILD ANOTHER

Los artículos de Wikipedia están relacionados mediante links. Para mantener esta información vamos a crear otra tabla:

```
hbase> create 'links', {
NAME => 'to', VERSIONS => 1, BLOOMFILTER => 'ROWCOL'
},{
NAME => 'from', VERSIONS => 1, BLOOMFILTER => 'ROWCOL'
}
```

Tiene sentido separar las tablas, en lugar de crear más family columns porque, al estar separadas por regiones, el cluster puede separar estas regiones más fácilmente.

Por este motivo se tiende a separar las tablas cuando sea posible y crear pocas familias.

### CONSTRUCTING THE SCANNER

Ahora vamos a construir un script que escanee la tabla wiki y la procese para extraer la información:

#### hbase/generate\_wiki\_links.rb

```
import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'
import 'org.apache.hadoop.hbase.client.Scan'
import 'org.apache.hadoop.hbase.util.Bytes'

def jbytes(*args)
  return args.map { |arg| arg.to_s.to_java_bytes }
end

wiki_table = HTable.new(@hbase.configuration, 'wiki')
links_table = HTable.new(@hbase.configuration, 'links')
links_table.setAutoFlush(false)

scanner = wiki_table.getScanner(Scan.new)

linkpattern = /\[\[([^\[\]\|\\:|#][^\[\]\|\\:]*)(?:\[([^\[\]\|\\]+))?\]\]\]/
count = 0

while (result = scanner.next())
  title = Bytes.toString(result.getRow())
  text = Bytes.toString(result.getValue(*jbytes('text', '')))
  if text
    put_to = nil
    text.scan(linkpattern) do |target, label|
      unless put_to
        put_to = Put.new(*jbytes(title))
        put_to.setWriteToWAL(false)
      end

      target.strip!
      target.capitalize!

      label = '' unless label
      label.strip!

      put_to.add(*jbytes("to", target, label))
      put_from = Put.new(*jbytes(target))
      put_from.add(*jbytes("from", title, label))
      put_from.setWriteToWAL(false)
      links_table.put(put_from)
    end
    links_table.put(put_to) if put_to
    links_table.flushCommits()
  end

  count += 1
  puts "#{count} pages processed (#{title})" if count % 500 == 0
end

links_table.flushCommits()
exit
```

## RUNNING THE SCRIPT

Para correr el script:

```
${HBASE_HOME}/bin/hbase shell generate_wiki_links.rb
```

### EXAMINING THE OUTPUT

Para ver los tipos de links que están siendo creados, escaneamos la tabla:

```
hbase> scan 'links', STARTROW => "Admiral Ackbar", ENDROW  
=> "It's a Trap!"
```

Se puede usar el comando get para ver los links de un único artículo:

```
hbase> get 'links', 'Addition'
```

Para saber cuántas filas hay en mi tabla podemos usar:

```
hbase> count 'wiki', INTERVAL => 100000, CACHE => 10000
```